Assignment No. 9

Name: Shubham Shivaji Phad
Class & Division: LY AIA 03 Batch B
Roll-No:48
Enrollment Number: ADT23SOCB1106

Title

Design and evaluate language models using N-Gram techniques and Neural Language Models to predict the next word in a sequence and assess language generation capabilities.

Aim

To design and evaluate N-Gram and Neural Language Models for next-word prediction and basic language generation.

Objectives

- Understand the concept of statistical N-Gram language models.
- Predict the next word using a bigram model.
- Build a simple neural language model using an LSTM network.
- Compare N-Gram and neural model predictions.
- Generate text from a given seed sequence.

Theory

A language model assigns probabilities to sequences of words and can predict the next word from previous context. An N-Gram model estimates the probability of a word using a fixed number of previous words. A bigram model uses one previous word, while a trigram model uses two.

Neural language models learn distributed word representations and use a neural network to predict the next word. An LSTM can retain useful information from previous words and is suitable for sequence prediction. Model performance can be assessed using prediction accuracy and language generation quality.

Software/Libraries Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Python collections
- TensorFlow / Keras
- NumPy

Installation Commands

```
!pip install tensorflow numpy -q
```

Algorithm/Steps

1. Prepare a small text corpus and convert it into lowercase tokens.
2. Build bigram counts and calculate conditional word probabilities.
3. Predict the most probable next word using the N-Gram model.
4. Create input sequences and target words for neural training.
5. Train an LSTM language model using an embedding layer and softmax output.
6. Predict the next word from a seed sequence.
7. Generate additional words repeatedly and compare the generated output.

```
from collections import Counter, defaultdict
import numpy as np

corpus = """
natural language processing helps computers understand human language
language models predict the next word in a sequence
neural language models learn patterns from text data
natural language models can generate meaningful text
deep learning improves language generation and text prediction
language generation is useful in many natural language applications
""".lower().split()

# Bigram language model
bigram_counts = defaultdict(Counter)
for w1, w2 in zip(corpus[:-1], corpus[1:]):
    bigram_counts[w1][w2] += 1

def ngram_predict(word):
    if word not in bigram_counts:
        return None
    return bigram_counts[word].most_common(1)[0][0]

print("N-Gram Next-Word Prediction")
for word in ["language", "models", "natural", "text"]:
    print(f"{word} -> {ngram_predict(word)}")
```

```
N-Gram Next-Word Prediction
language -> models
models -> predict
natural -> language
text -> data
```

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

tokenizer = Tokenizer()
tokenizer.fit_on_texts([corpus])
total_words = len(tokenizer.word_index) + 1

sequences = []
for line in [
    "natural language processing helps computers understand human language",
    "language models predict the next word in a sequence",
    "neural language models learn patterns from text data",
    "natural language models can generate meaningful text",
    "deep learning improves language generation and text prediction",
    "language generation is useful in many natural language applications"
]:
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        sequences.append(token_list[:i+1])

max_len = max(len(s) for s in sequences)
sequences = pad_sequences(sequences, maxlen=max_len, padding='pre')
X = sequences[:, :-1]
y = tf.keras.utils.to_categorical(sequences[:, -1], num_classes=total_words)

model = Sequential([
    Embedding(total_words, 32, input_length=max_len-1),
    LSTM(64),
    Dense(total_words, activation='softmax')
])
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=
        ['accuracy'])
model.fit(X, y, epochs=100, verbose=0)

print("Neural Language Model trained successfully.")
print("Training accuracy:", round(float(model.evaluate(X, y, verbose=0)[1]),
        4))
```

```
/usr/local/lib/python3.13/dist-
packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument
`input_length` is deprecated. Just remove it.
  warnings.warn(

Neural Language Model trained successfully.
Training accuracy: 0.7674
```

```
def neural_predict(seed_text):
    seq = tokenizer.texts_to_sequences([seed_text.lower()])[0]
    seq = pad_sequences([seq], maxlen=max_len-1, padding='pre')
    probs = model.predict(seq, verbose=0)[0]
    index = int(np.argmax(probs))
    return tokenizer.index_word.get(index, "<unknown>")

def generate_text(seed_text, n_words=6):
    text = seed_text
    for _ in range(n_words):
```

```
        next_word = neural_predict(text)
        text += " " + next_word
    return text

print("Neural Next-Word Prediction")
for seed in ["natural language", "language models", "deep learning"]:
    print(f"{seed} -> {neural_predict(seed)}")

print("\nGenerated Text")
print(generate_text("natural language", 6))
```

```
Neural Next-Word Prediction
natural language -> models
language models -> models
deep learning -> language

Generated Text
natural language models models generate meaningful text text
```

Program with Output

The program produces next-word predictions from both models and generates a short sequence using the neural language model.

Typical output:

- N-Gram prediction displays the most frequent word following the given context.
- The LSTM model displays its training accuracy and predicts the next word from a seed sequence.
- Generated text demonstrates the language generation capability of the neural model.

Training accuracy and generated text can vary slightly because neural model training is stochastic.

Conclusion

N-Gram and Neural Language Models were successfully implemented for next-word prediction. The N-Gram model uses local word frequency patterns, while the LSTM model learns sequence patterns through neural representations. The experiment demonstrates how both approaches can be used for language prediction and basic text generation.